CS433: Assignment#2 (Due by 28th October 2026 midnight)

1. In this problem, you will implement a lock-free semaphore that can only use the `cmpxchg` instruction for implementing atomic lock-free operations. It should not use custom locks built using the `cmpxchg` instruction or any other locks. The semaphore should have the following interface functions.

- `initialize`: Initializes the semaphore.
- `wait`: Implements semaphore wait call using busy-waiting. This functionality is similar to Linux semaphore's `sem_wait` function except that here the calling thread is not context-switched.
- `post`: Implements semaphore post call. This functionality is similar to Linux semaphore's `sem_post` function.
- `destroy`: Frees any internally allocated memory of the semaphore.

The implemented semaphore must guarantee that the waiting threads are granted access to the protected resource in a FIFO order. You should feel free to use the `CompareAndSet` function developed in the class. You will evaluate the semaphore using a multi-producer single-consumer program. First, upgrade the single-producer single-consumer program discussed in the class to support multiple producers and one consumer. The program should not use any locks, but can use lock-free atomic operations implemented using the `cmpxchg` instruction. The producers produce values into empty slots of an array, while the consumer reads produced values from the array slots and prints them. A producer must not overwrite an array slot that has not yet been read. The program should take the array size (N), the number of producers (P), and the number of values produced by each producer (n) from the command line in that order. The output of the program is the list of nP values consumed by the consumer. Implement the following three versions of this program.

1. Program P1: uses your lock-free semaphore.
2. Program P2: uses Linux semaphore.
3. Program P3: uses condition variables. In this case, you are allowed to use POSIX locks for protecting the condition variables only. Do not introduce any other locks.

Compare the performance of P1, P2, P3 for a range of values of N, P, n. Make sure to choose N and n such that the program runs for at least few hundred milliseconds. The larger they are, the better it is in averaging out the measurement errors. Sweep P as permitted by the underlying hardware. **(50 points)**

2. A directed graph G can be represented using an array of linked lists where list#i stores the vertex ids j such that (i, j) is an edge in G. In this problem, you will implement a directed graph with a given number of vertices and edges using an array of linked lists. The graph must support three operations, namely, addition of an edge, deletion of an edge, and search for an edge. Initially, a graph having n vertices and m edges is built. A list of operations having a mix of the aforementioned three operations is executed concurrently by a group of threads. Each operation in the operation list contains an operation code (insert, delete, or search) and a pair of randomly generated distinct vertex ids u and v representing the edge (u, v) to be operated on. The program should read the operation list from a file. The program takes the operation file name, the number of operations, and thread count from the command line in that order. Implement the following versions of the program.

1. Program P1: uses a single POSIX mutex lock for the entire graph.

2. Program P2: uses a POSIX mutex lock per list.

3. Program P3: uses a POSIX mutex lock per list node.

4. Program P4: uses a POSIX read/write lock per list node.

5. Program P5: uses a custom lock per list node based on the `cmpxchg` instruction as developed in the class.

6. Program P6: uses optimistic locking scheme with a custom lock per list node based on the `cmpxchg` instruction as developed in the class.

Compare the performance of P1, P2, P3, P4, P5, P6 for a range of thread count, three different ratios of operations, and three different total number of operations. Pick sufficiently large operation counts to average out measurement errors. Fix the three different operation ratios as (80% insert, 10% delete, 10% search), (30% insert, 30% delete, 40% search), and (10% insert, 10% delete, 80% search). The operations should be uniformly mixed throughout the operation list. Please refer to the similar example code discussed in the class. Keep the initial graph fixed in all experiments. Pick a sufficiently large initial graph which is not complete. **(50 points)**

What to submit and how. For each question, submit the parallel program. Prepare a report describing your parallel algorithms, the optimizations you applied (if any) for improving performance, and the performance results. Note that your marks will depend on how good your program is in terms of performance, how well the algorithm and the program are described in the report, and the discussion on the results. In your measured time, do not include any time that is needed to allocate memory or initialize data structures. Please refer to the examples discussed in the class. You will not get any marks if your program does not compile or crashes during execution. Only the programs that compile and run to completion without any error will be considered for grading. Please do not use any external library outside POSIX threads and standard C/C++. That will make testing your code very difficult. Please feel free to refer to and cite papers/books, but do not copy codes from other places. Please send your submission (two programs and a PDF report) via email to mainakc@cse.iitk.ac.in with subject "[CS433] Assignment#2 submission Group#X". Replace X by your group number in the subject line.